Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Уфимский университет науки и технологий»

Кафедра: Высокопроизводительных вычислений и дифференциальных уравнений

КОМПЬЮТЕРНОЕ МОДЕЛИРОВАНИЕ ДВИЖЕНИЯ КОСМИЧЕСКИХ ТЕЛ

ОТЧЕТ

к лабораторной работе №1 по дисциплине

«Математическое моделирование»

Группа МКН-418Б	Фамилия И. О.	Подпись	Дата	Оценка
Студент	Халитова А.А.			
Принял	Феоктистов Б.А.			

Уфа 2026

Содержание

1 Ведение **3**

2 Теоретическая часть **4**
2.1 Задача I 4
2.2 Задача II 4

3 Практическая часть **5**
3.1 Задача I 5
3.2 Задача II 8

4 Вывод **11**

5 Листинг **12**

1 Ведение

Цель работы: получить навык численного расчета траекторий движения космических тел под действием гравитационных сил.

2 Теоретическая часть

2.1 Задача I

Рассматривается динамика трех разновеликих небесных тел: звезды, планеты и ее спутника. В качестве примера рассматривается Солнечная система. Масса Солнца $M_1 = 2 \cdot 10^{30}$. Параметры двух других тел выбираются в соответствии с номером варианта из таблицы.

1. Составить уравнения движения второго и третьего тела в системе отсчета, связанной с первым (самым массивным) телом. Предполагается, что движение всех тел происходит в одной плоскости.

2. Написать программу численного интегрирования составленных уравнений движения и построить траектории движения тел. В качестве начальных условий принять следующие: все тела находятся на одной прямой, векторы скоростей движения второго и третьего тела сонаправлены. Расстояния между первым и вторым, а также вторым и третьим телами приведены в таблице. Там же указаны значения начальных скоростей второго и третьего тела.

2.2 Задача II

На круговой орбите высотой H второго тела находится космический корабль. В некоторый момент времени его двигатели включаются и работают в течение времени T, выводя корабль на новую орбиту, пересекающую орбиту третьего тела. Вектор тяги двигателя в любой момент времени направлен по касательной к траектории движения. Определить местоположение космического корабля на первоначальной орбите в момент включения двигателя из условия минимума массы топлива, необходимой для доставки на поверхность третьего тела полезного груза массой M_0. Местоположение определяется относительно прямой, соединяющей центры второго и третьего тел. Масса корабля складывается из массы топлива, полностью выгорающего за время T, массы конструкции (0.025 стартовой массы), массы полезной нагрузки M_0. В конце активного участка траектории (через время T) происходит отделение полезного груза, который движется далее только под действием гравитационных сил. Скорость полезного груза при достижении поверхности третьего тела не ограничивается.

3 Практическая часть

3.1 Задача I

Задание выполнялось с параметрами для 7 варианта, приведёнными в таблице 1.

Таблица 1 — Параметры небесных тел

№ варианта	Параметры второго тела				Параметры третьего тела			
	M_2, кг	R_2, км	R_{12}, млн км	V_2, км/с	M_3, кг	R_3, км	R_{23}, тыс. км	V_3, км/с
7	$1.9 \cdot 10^{26}$	71500	780	13	$4.8 \cdot 10^{22}$	1561	671	13.7

В системе действует сила притяжения космических тел, которая подчиняется закону всемирного тяготения:

$$\vec{F}_{ij} = G\frac{m_i m_j}{|r_{ij}|^3}\vec{r}_{ij},$$

где $G = 6.67 \times 10^{-11}$ м^3/кг·с^2 — гравитационная постоянная, $m_i j$ — массы взаимодействующих тел, $\vec{r}_{ij}$ — радиус-вектор.

Уравнения движения:

$$M_i\frac{\partial \vec{v}_i}{\partial t} = \sum_{j=1}^{3}\vec{F}_{ij}, \vec{v}_i = \frac{\partial \vec{r}_i}{\partial t}.$$

Общая система ОДУ будет иметь вид:

$$\begin{cases} M_2\dfrac{d\vec{v}_2}{dt} = -GM_1M_2\dfrac{\vec{r}_{12}}{|\vec{r}_{12}|^3} + GM_2M_3\dfrac{\vec{r}_{23}}{|\vec{r}_{23}|^3}, \\ M_3\dfrac{d\vec{v}_3}{dt} = -GM_1M_3\dfrac{\vec{r}_{13}}{|\vec{r}_{13}|^3} - GM_2M_3\dfrac{\vec{r}_{23}}{|\vec{r}_{23}|^3}, \\ \dfrac{d\vec{r}_{12}}{dt} = \vec{v}_2, \\ \dfrac{d\vec{r}_{13}}{dt} = \vec{v}_3, \end{cases}$$

где радиус-векторы определяются:

$$|\vec{r}_{12}| = \sqrt{x_2^2 + y_2^2},$$
$$|\vec{r}_{13}| = \sqrt{x_3^2 + y_3^2},$$
$$|\vec{r}_{23}| = \sqrt{(x_3 - x_2)^2 + (y_3 - y_2)^2}.$$

Начальные условия:

$$\begin{cases} \vec{r}_2(0) = (R_{12},\ 0), \\ \vec{r}_3(0) = (R_{12} + R_{23},\ 0), \\ \vec{v}_2(0) = (0,\ V_2), \\ \vec{v}_3(0) = (0,\ V_2 + V_3), \end{cases}$$

Задача Коши решалась методом Рунге-Кутты 4 порядка на языке C++. Полученные траектории движения планеты и её спутника показаны на рисунке 1.

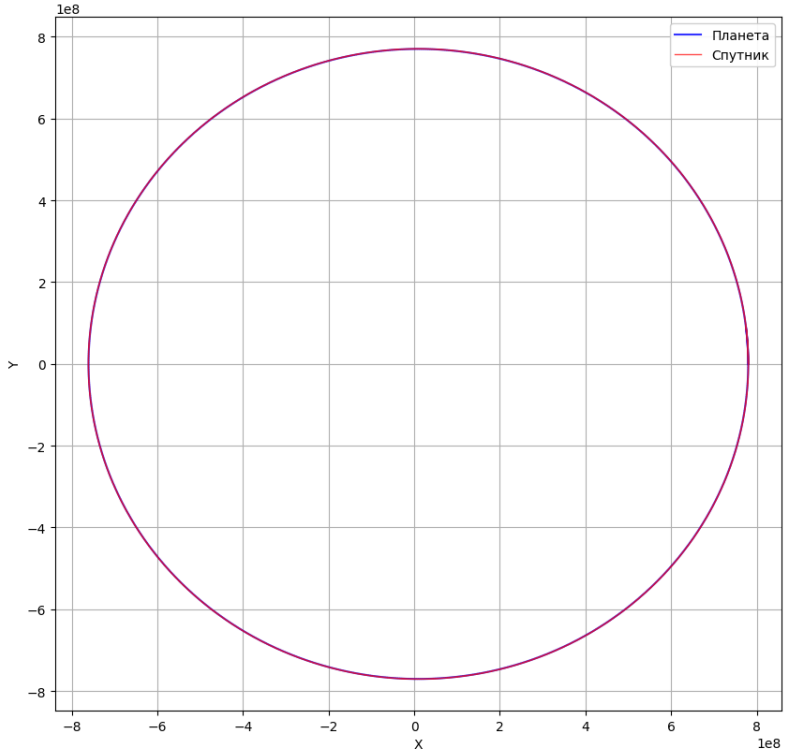

Рисунок 1 — Траектории движения планеты и спутника вокруг Солнца

Траектория движения спутника напоминает синусоиду. Более детально она показана на рисунке 2.

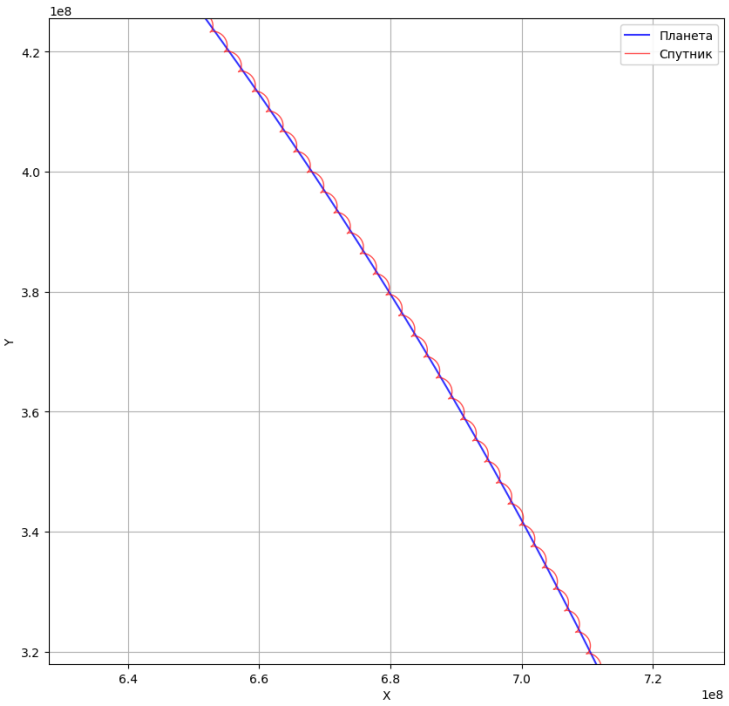

Рисунок 2 — Траектория движения спутника относительно планеты

Орбита спутника не замкнута — после каждого оборота планеты вокруг Солнца он не возвращается в исходную точку. На рисунке 3 показана траектория спутника за 11 оборотов планеты вокруг Солнца.

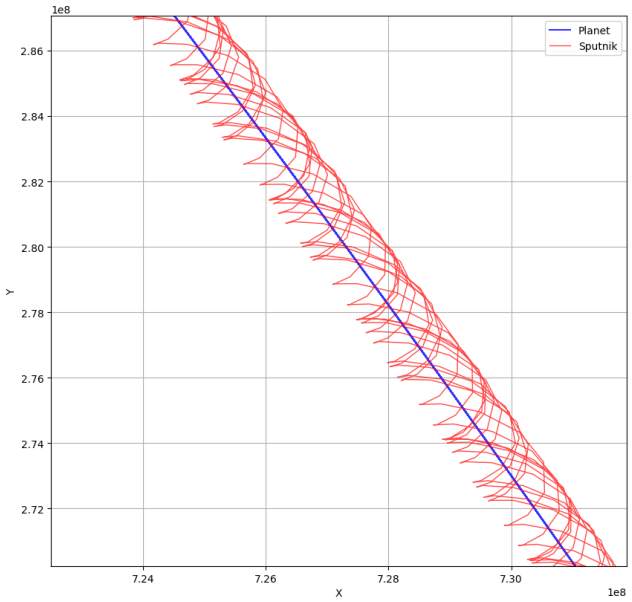

Рисунок 3 — Траектория спутника за 11 оборотов

На рисунке 4 показаны траектории движения планеты за 11 оборотов

вокруг солнца.

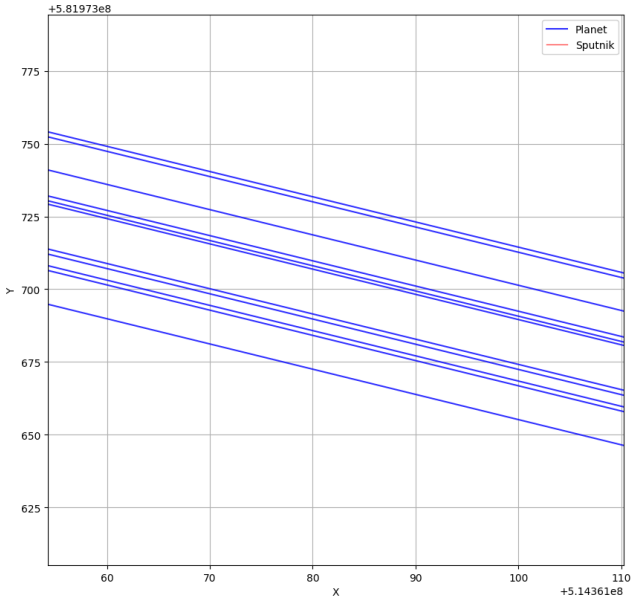

Рисунок 4 — Траектория планеты за 11 оборотов

Период обращения планеты вокруг Солнца вычислялся по третьему закону Кеплера:

$$T = 2\pi\sqrt{\frac{|\vec{r}_{12}|^3}{GM_1}}.$$

3.2 Задача II

Параметры для решения задачи приведены в таблице 2.

Таблица 2 — Параметры задачи 2

№ варианта	H, км	T, с	M_0, кг	Характеристики топлива		
				Горючее	Окислитель	Скорость истечения, м/с
7	2000	5100	40	НДМГ	Кислород (жидкий)	3375

Для описания движения точки переменной массы воспользуемся уравнением Мещерского:

$$m\frac{d\vec{v}}{dt} = -\vec{u}\frac{dm}{dt} + \sum_{i=1}^{3}\vec{F}_{ki},$$

где $\vec{v}$ – скорость ракеты, $\vec{u}$ – скорость истечения газов.

Общая масса ракеты складывается из трех компонент:

$$M_{\text{общая}} = M_{\text{к}} + M_{\text{т}} + M_0,$$

где $M_{\text{к}}$ – масса конструкции ракеты, $M_{\text{т}}$ – масса топлива, M_0 – масса полезной нагрузки. Доля конструкции определяется соотношением:

$$\frac{M_{\text{к}}}{M_{\text{общая}}} = 0{,}001.$$

Масса ракеты изменяется во времени по следующему закону:

$$m(t) = \begin{cases} \dfrac{M_0 + M_{\text{т}}}{1 - 0{,}001} - \dfrac{M_{\text{т}}t}{T}, & t < T, \\ M_0, & t \geq T. \end{cases}$$

где T – время работы двигателя.

В результате в систему обыкновенных дифференциальных уравнений добавляются уравнения для компонент скорости ракеты:

$$m(t)\frac{d\vec{v}(t)}{dt} = -u\frac{\vec{v}(t)}{|\vec{v}(t)|}\frac{dm(t)}{dt} - Gm(t)\left(\frac{M_1\vec{r}(t)}{|\vec{r}(t)|^3} + \frac{M_2\vec{r}_2(t)}{|\vec{r}_2(t)|^3} + \frac{M_3\vec{r}_3(t)}{|\vec{r}_3(t)|^3}\right),$$

где $\vec{r}(t) = \big(x(t), y(t)\big)$ — положение ракеты относительно звезды, $\vec{r}_2(t) = (\vec{r}(t) - \vec{r}_2(t))$ — положение ракеты относительно планеты, $\vec{r}_3(t) = (\vec{r}(t) - \vec{r}_3(t))$ — положение ракеты относительно спутника.

Начальное положение и скорость ракеты на орбите заданной высоты:

$$\begin{cases} x(0) = x_2 + (R_2 + H)\cdot\cos\alpha, \\ y(0) = y_2 + (R_2 + H)\cdot\sin\alpha, \\ v_x(0) = v_{2x} - V_{1\text{к}}\cdot\sin\alpha, \\ v_y(0) = v_{2y} + V_{1\text{к}}\cdot\cos\alpha, \end{cases}$$

где $V_{1\text{к}} = \sqrt{\dfrac{GM_2}{R_2 + H}}$ — первая космическая скорость, (v_{2x}, v_{2y}) - скорость планеты.

На рисунке 5 показана траектория движения ракеты и спутника вокруг планеты до столкновения.

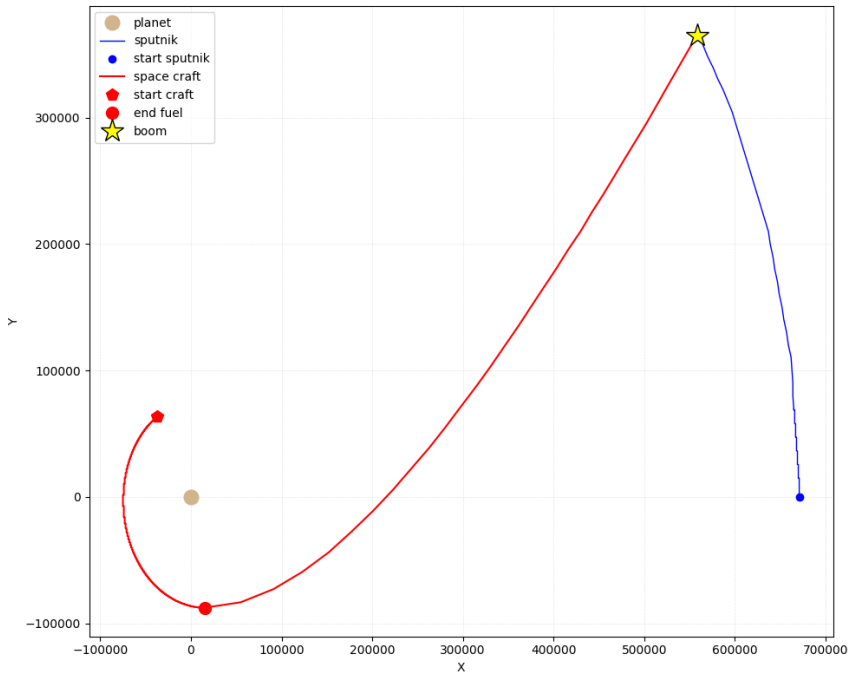

Рисунок 5 — Траектория движения космического аппарата и спутника до столкновения

В результате численного моделирования определены оптимальные параметры старта, обеспечивающие столкновение с поверхностью спутника:

$$M_T = 39\,867.11 \text{ кг},$$
$$\alpha = 119.9^\circ.$$

4 Вывод

В ходе выполнения работы получены навыки численного расчета траекторий движения космических тел под действием гравитационных сил.

Реализован метод Рунге-Кутты 4-го порядка на языке C++ для моделирования системы из трёх тел (Солнце, планета, спутник) и выполнен поиск оптимальных параметров запуска ракеты для её столкновения со спутником.

5 Листинг

main.cpp

```cpp
#include <chrono>
#include <iostream>
#include <memory>

#include "solver.h"
#include "spacecraft_solver.h"

using namespace std;

int main() {
        cout << "sun, planet, sputnik, part 1" << endl;

        auto start = std::chrono::steady_clock::now();
        bool add_spaace_apparat = false;
        int num_steps = 1000000;
        int num_years = 1;
        unique_ptr<Solver> task1 = make_unique<Solver>(num_steps, num_years);
        task1->run_computations();
        auto end = std::chrono::steady_clock::now();

        std::chrono::duration<double, std::milli> duration = end - start;
        cout << "time of 1 part computations = " << duration.count() << endl;

        task1->save_to_csv("../labs/lr1/src/trajectory_full.csv");
        cout << "haha" << endl;
        system("python ../labs/lr1/src/main.py");

        cout << "space apparat part 2" << endl;

        auto start_2 = std::chrono::steady_clock::now();
        int num_steps_burn = 1000000;
        int num_steps_coast = 1000000;

        auto task2 = make_unique<SpaceCraftSolver>(
        &task1->get_trajectory(), task1->get_time_to_around(), num_steps_burn, num_steps_coast);

        bool use_parallel = true;
        task2->run_computations(use_parallel);

        auto end_2 = std::chrono::steady_clock::now();
        std::chrono::duration<double> total_time = end_2 - start_2;

        cout << "time of 2 part computations = " << total_time.count() << " s" << endl;
        system("python ../labs/lr1/src/part.py");

        return 0;
}
```

solver.h

```cpp
#pragma once
#include <string>
#include <vector>
#include "state.h"

using namespace std;

class Solver {
        private:
        pair<double, double> sun_initial_pos, planet_initial_pos, sputnik_initial_pos;
        pair<double, double> v_planet, v_sputnik;
        double time_to_around;
        int num_steps;
        double step;
        vector<State> trajectory;

        public:
        Solver(int num_steps, int num_years);

        vector<State>& get_trajectory();
        double get_time_to_around();

        double dist(double x, double y);
        void compute_derivatives(const State& s, pair<double, double>& d_planet_pos,
        pair<double, double>& d_sputnik_pos,
        pair<double, double>& d_v_planet,
        pair<double, double>& d_v_sputnik);
        void runge_step(State& s, double dt);
        void run_computations();
        void save_to_csv(const string& filename) const;
};
```

solver.cpp

```cpp
#include "solver.h"

#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>

#include "consts.h"

using namespace std;

Solver::Solver(int num_steps, int num_years) : num_steps(num_steps) {
        sun_initial_pos = {0.0, 0.0};
        planet_initial_pos = {Constants::R_SUN_PLANET, 0.0};
        sputnik_initial_pos = {Constants::R_SUN_PLANET + Constants::R_PLANET_SPUTNIK, 0.0};

        v_planet = {0.0, Constants::V_PLANET};
        v_sputnik = {0.0, Constants::V_PLANET + Constants::V_SPUTNIK};

        time_to_around = 2 * M_PI *
```

```
        sqrt(Constants::R_SUN_PLANET * Constants::R_SUN_PLANET *
        Constants::R_SUN_PLANET / (Constants::G * Constants::M_SUN));
        cout << "time␣to␣make␣a␣circle␣around␣sun␣=␣" << time_to_around << endl;

        step = num_years * time_to_around / num_steps;
}

vector<State>& Solver::get_trajectory() { return trajectory; }

double Solver::get_time_to_around() { return time_to_around; }

double Solver::dist(double x, double y) { return sqrt(x * x + y * y); }

void Solver::compute_derivatives(const State& s, pair<double, double>& d_planet_pos,
pair<double, double>& d_sputnik_pos,
pair<double, double>& d_v_planet,
pair<double, double>& d_v_sputnik) {
        double dx_sputnik_planet = s.sputnik_pos.first - s.planet_pos.first;
        double dy_sputnik_planet = s.sputnik_pos.second - s.planet_pos.second;

        double r_sun_sputnik = dist(s.sputnik_pos.first, s.sputnik_pos.second);
        double r_sun_planet = dist(s.planet_pos.first, s.planet_pos.second);
        double r_planet_sputnik = dist(dx_sputnik_planet, dy_sputnik_planet);

        d_planet_pos.first = s.v_planet.first;
        d_planet_pos.second = s.v_planet.second;
        d_sputnik_pos.first = s.v_sputnik.first;
        d_sputnik_pos.second = s.v_sputnik.second;

        double ax_planet_sun = -Constants::G * Constants::M_SUN * s.planet_pos.first /
        (r_sun_planet * r_sun_planet * r_sun_planet);
        double ay_planet_sun = -Constants::G * Constants::M_SUN * s.planet_pos.second /
        (r_sun_planet * r_sun_planet * r_sun_planet);

        double ax_planet_sputnik = Constants::G * Constants::M_SPUTNIK * dx_sputnik_planet /
        (r_planet_sputnik * r_planet_sputnik * r_planet_sputnik);
        double ay_planet_sputnik = Constants::G * Constants::M_SPUTNIK * dy_sputnik_planet /
        (r_planet_sputnik * r_planet_sputnik * r_planet_sputnik);

        double ax_sputnik_sun = -Constants::G * Constants::M_SUN * s.sputnik_pos.first /
        (r_sun_sputnik * r_sun_sputnik * r_sun_sputnik);
        double ay_sputnik_sun = -Constants::G * Constants::M_SUN * s.sputnik_pos.second /
        (r_sun_sputnik * r_sun_sputnik * r_sun_sputnik);

        double ax_sputnik_planet = -Constants::G * Constants::M_PLANET *
```

```
            dx_sputnik_planet /
        (r_planet_sputnik * r_planet_sputnik * r_planet_sputnik);
        double ay_sputnik_planet = -Constants::G * Constants::M_PLANET *
            dy_sputnik_planet /
        (r_planet_sputnik * r_planet_sputnik * r_planet_sputnik);

        d_v_planet.first = ax_planet_sun + ax_planet_sputnik;
        d_v_planet.second = ay_planet_sun + ay_planet_sputnik;
        d_v_sputnik.first = ax_sputnik_sun + ax_sputnik_planet;
        d_v_sputnik.second = ay_sputnik_sun + ay_sputnik_planet;
}

void Solver::runge_step(State& s, double dt) {
        pair<double, double> k1_planet_pos, k1_sputnik_pos, k1_v_planet,
            k1_v_sputnik;
        pair<double, double> k2_planet_pos, k2_sputnik_pos, k2_v_planet,
            k2_v_sputnik;
        pair<double, double> k3_planet_pos, k3_sputnik_pos, k3_v_planet,
            k3_v_sputnik;
        pair<double, double> k4_planet_pos, k4_sputnik_pos, k4_v_planet,
            k4_v_sputnik;

        State temp;

        compute_derivatives(s, k1_planet_pos, k1_sputnik_pos,
            k1_v_planet, k1_v_sputnik);

        temp = s;
        temp.planet_pos.first += k1_planet_pos.first * dt / 2;
        temp.planet_pos.second += k1_planet_pos.second * dt / 2;
        temp.sputnik_pos.first += k1_sputnik_pos.first * dt / 2;
        temp.sputnik_pos.second += k1_sputnik_pos.second * dt / 2;
        temp.v_planet.first += k1_v_planet.first * dt / 2;
        temp.v_planet.second += k1_v_planet.second * dt / 2;
        temp.v_sputnik.first += k1_v_sputnik.first * dt / 2;
        temp.v_sputnik.second += k1_v_sputnik.second * dt / 2;
        temp.time = s.time + dt / 2;
        compute_derivatives(temp, k2_planet_pos, k2_sputnik_pos,
            k2_v_planet, k2_v_sputnik);

        temp = s;
        temp.planet_pos.first += k2_planet_pos.first * dt / 2;
        temp.planet_pos.second += k2_planet_pos.second * dt / 2;
        temp.sputnik_pos.first += k2_sputnik_pos.first * dt / 2;
        temp.sputnik_pos.second += k2_sputnik_pos.second * dt / 2;
        temp.v_planet.first += k2_v_planet.first * dt / 2;
        temp.v_planet.second += k2_v_planet.second * dt / 2;
        temp.v_sputnik.first += k2_v_sputnik.first * dt / 2;
        temp.v_sputnik.second += k2_v_sputnik.second * dt / 2;
        temp.time = s.time + dt / 2;
        compute_derivatives(temp, k3_planet_pos, k3_sputnik_pos,
            k3_v_planet, k3_v_sputnik);

        temp = s;
        temp.planet_pos.first += k3_planet_pos.first * dt;
        temp.planet_pos.second += k3_planet_pos.second * dt;
        temp.sputnik_pos.first += k3_sputnik_pos.first * dt;
        temp.sputnik_pos.second += k3_sputnik_pos.second * dt;
```

```
        temp.v_planet.first += k3_v_planet.first * dt;
        temp.v_planet.second += k3_v_planet.second * dt;
        temp.v_sputnik.first += k3_v_sputnik.first * dt;
        temp.v_sputnik.second += k3_v_sputnik.second * dt;
        temp.time = s.time + dt;
        compute_derivatives(temp, k4_planet_pos, k4_sputnik_pos,
            k4_v_planet, k4_v_sputnik);

        s.planet_pos.first += (k1_planet_pos.first + 2 *
            k2_planet_pos.first + 2 * k3_planet_pos.first +
        k4_planet_pos.first) *
        dt / 6;
        s.planet_pos.second += (k1_planet_pos.second + 2 *
            k2_planet_pos.second +
        2 * k3_planet_pos.second + k4_planet_pos.second) *
        dt / 6;
        s.sputnik_pos.first += (k1_sputnik_pos.first + 2 *
            k2_sputnik_pos.first +
        2 * k3_sputnik_pos.first + k4_sputnik_pos.first) *
        dt / 6;
        s.sputnik_pos.second += (k1_sputnik_pos.second + 2 *
            k2_sputnik_pos.second +
        2 * k3_sputnik_pos.second + k4_sputnik_pos.second) *
        dt / 6;

        s.v_planet.first +=
        (k1_v_planet.first + 2 * k2_v_planet.first + 2 *
            k3_v_planet.first + k4_v_planet.first) *
        dt / 6;
        s.v_planet.second += (k1_v_planet.second + 2 *
            k2_v_planet.second + 2 * k3_v_planet.second +
        k4_v_planet.second) *
        dt / 6;
        s.v_sputnik.first += (k1_v_sputnik.first + 2 *
            k2_v_sputnik.first + 2 * k3_v_sputnik.first +
        k4_v_sputnik.first) *
        dt / 6;
        s.v_sputnik.second += (k1_v_sputnik.second + 2 *
            k2_v_sputnik.second + 2 * k3_v_sputnik.second +
        k4_v_sputnik.second) *
        dt / 6;

        s.time += dt;
}

void Solver::run_computations() {
        State current(sun_initial_pos, planet_initial_pos,
            sputnik_initial_pos, v_planet, v_sputnik);
        trajectory.clear();
        trajectory.push_back(current);

        for (int i = 0; i <= num_steps; i++) {
                runge_step(current, step);
                if (i % 10 == 0) {
                        trajectory.push_back(current);
                }
        }
}
```

```

void Solver::save_to_csv(const string& filename) const {
        ofstream file(filename);

        file << "time,x_planet,y_planet,x_sputnik,y_sputnik,"
        << "vx_planet,vy_planet,vx_sputnik,vy_sputnik" << endl;

        file << scientific << setprecision(20);

        for (const auto& s : trajectory) {
                file << s.time << "," << s.planet_pos.first << "," << s.planet_pos.second << ","
                << s.sputnik_pos.first << "," << s.sputnik_pos.second << "," << s.v_planet.first << ","
                << s.v_planet.second << "," << s.v_sputnik.first << "," << s.v_sputnik.second << endl;
        }

        file.close();
}
```

spacecraftSolver.h

```
#pragma once

#include <string>
#include <vector>

#include "state.h"

using namespace std;

class SpaceCraftSolver {
        private:
        vector<State>* base_traj;
        double planet_period;
        double orbit_radius;
        int steps_burn, steps_coast;
        double dt_burn, dt_coast;
        double best_mass, best_angle, best_length;

        double norm(const pair<double, double>& v);
        pair<double, double> gravity_acc(const SpaceCraftState& craft, double t);
        State get_state_at_time(double t) const;
        SpaceCraftState compute_derivatives(const SpaceCraftState& craft, double t, bool active,
        double fuel_rate);
        SpaceCraftState runge_step(const SpaceCraftState& sc, double t, double dt, bool active,
        double fuel_rate);
        SpaceCraftState init_space_craft(double angle_deg, double mass);
        double try_flight(double angle, double mass, double total_time);
        void find_best_fuel_angle_omp(double total_time);
        void find_best_fuel_angle(double total_time);

        public:
        SpaceCraftSolver(vector<State>* traj, double period, int n_burn,
```

```
            int n_coast);

        void run_computations(bool type_of_finding);
        void save_to_csv(double total_time);
};
```

spacecraftSolver.cpp

```
#include "spacecraft_solver.h"

#include <omp.h>

#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>

#include "consts.h"
#include "solver.h"

using namespace std;

SpaceCraftSolver::SpaceCraftSolver(vector<State>* traj, double period, int n_burn, int n_coast)
: base_traj(traj), planet_period(period), orbit_radius(Constants::R_PLANET + Constants::H) {
        steps_burn = n_burn;
        steps_coast = n_coast;
        dt_burn = Constants::T_BURN / steps_burn;

        planet_period =
        2.0 * M_PI *
        sqrt(pow(Constants::R_PLANET_SPUTNIK, 3) / (Constants::G * Constants::M_PLANET));
        dt_coast = (2.0 * period) / steps_coast;
}

double SpaceCraftSolver::norm(const pair<double, double>& v) {
        return sqrt(v.first * v.first + v.second * v.second);
}

pair<double, double> SpaceCraftSolver::gravity_acc(const SpaceCraftState& craft, double t) {
        State sys = get_state_at_time(t);

        double rsx = craft.craft_pos.first;
        double rsy = craft.craft_pos.second;
        double rpx = craft.craft_pos.first - sys.planet_pos.first;
        double rpy = craft.craft_pos.second - sys.planet_pos.second;
        double rspx = craft.craft_pos.first - sys.sputnik_pos.first;
        double rspy = craft.craft_pos.second - sys.sputnik_pos.second;

        double rs = norm({rsx, rsy});
        double rp = norm({rpx, rpy});
        double rsp = norm({rspx, rspy});

        const double eps = 1e-6;
        rs = max(rs, eps);
```

```
        rp = max(rp, eps);
        rsp = max(rsp, eps);

        double ax = -Constants::G * Constants::M_SUN * rsx / (rs * rs * rs) -
        Constants::G * Constants::M_PLANET * rpx / (rp * rp * rp) -
        Constants::G * Constants::M_SPUTNIK * rspx / (rsp * rsp * rsp);
        double ay = -Constants::G * Constants::M_SUN * rsy / (rs * rs * rs) -
        Constants::G * Constants::M_PLANET * rpy / (rp * rp * rp) -
        Constants::G * Constants::M_SPUTNIK * rspy / (rsp * rsp * rsp);

        return {ax, ay};
}

State SpaceCraftSolver::get_state_at_time(double t) const {
        if (base_traj->empty()) {
                return State();
        }

        int left = 0;
        int right = static_cast<int>(base_traj->size()) - 1;
        while (right - left > 1) {
                int mid = (left + right) / 2;
                if ((*base_traj)[mid].time < t) {
                        left = mid;
                } else {
                        right = mid;
                }
        }

        const State& s1 = (*base_traj)[left];
        const State& s2 = (*base_traj)[right];
        double alpha = (t - s1.time) / (s2.time - s1.time);
        double beta = 1.0 - alpha;

        State res;
        res.sun_pos = s1.sun_pos;
        res.planet_pos = {beta * s1.planet_pos.first + alpha * s2.planet_pos.first,
                beta * s1.planet_pos.second + alpha * s2.planet_pos.second};
        res.sputnik_pos = {beta * s1.sputnik_pos.first + alpha * s2.sputnik_pos.first,
                beta * s1.sputnik_pos.second + alpha * s2.sputnik_pos.second};
        res.v_planet = {beta * s1.v_planet.first + alpha * s2.v_planet.first,
                beta * s1.v_planet.second + alpha * s2.v_planet.second};
        res.v_sputnik = {beta * s1.v_sputnik.first + alpha * s2.v_sputnik.first,
                beta * s1.v_sputnik.second + alpha * s2.v_sputnik.second};
        res.time = t;
        return res;
}

SpaceCraftState SpaceCraftSolver::compute_derivatives(const
```

```
    SpaceCraftState& craft, double t,
bool active, double fuel_rate) {
        pair<double, double> g_acc = gravity_acc(craft, t);
        SpaceCraftState deriv;
        deriv.craft_pos = craft.v_craft;
        deriv.time = craft.time;

        if (active) {
                double v_norm = norm(craft.v_craft);
                pair<double, double> thrust_dir;
                if (v_norm > 1e-10) {
                        thrust_dir = make_pair(craft.v_craft.first /
                            v_norm, craft.v_craft.second / v_norm);
                } else {
                        thrust_dir = make_pair(1.0, 0.0);
                }

                double thrust_acc = Constants::V_EXP * fuel_rate /
                    craft.mass_space_craft;
                deriv.v_craft = {g_acc.first + thrust_dir.first *
                    thrust_acc,
                        g_acc.second + thrust_dir.second * thrust_acc};
                deriv.mass_space_craft = -fuel_rate;
        } else {
                deriv.v_craft = g_acc;
                deriv.mass_space_craft = 0.0;
        }
        return deriv;
}

SpaceCraftState SpaceCraftSolver::runge_step(const SpaceCraftState& sc,
    double t, double dt,
bool active, double fuel_rate) {
        SpaceCraftState k1 = compute_derivatives(sc, t, active,
            fuel_rate);

        SpaceCraftState s2;
        s2.craft_pos = {sc.craft_pos.first + k1.craft_pos.first * dt *
            0.5,
                sc.craft_pos.second + k1.craft_pos.second * dt * 0.5};
        s2.v_craft = {sc.v_craft.first + k1.v_craft.first * dt * 0.5,
                sc.v_craft.second + k1.v_craft.second * dt * 0.5};
        s2.mass_space_craft = sc.mass_space_craft + k1.mass_space_craft
            * dt * 0.5;
        s2.time = sc.time + dt * 0.5;
        SpaceCraftState k2 = compute_derivatives(s2, t + dt * 0.5,
            active, fuel_rate);

        SpaceCraftState s3;
        s3.craft_pos = {sc.craft_pos.first + k2.craft_pos.first * dt *
            0.5,
                sc.craft_pos.second + k2.craft_pos.second * dt * 0.5};
        s3.v_craft = {sc.v_craft.first + k2.v_craft.first * dt * 0.5,
                sc.v_craft.second + k2.v_craft.second * dt * 0.5};
        s3.mass_space_craft = sc.mass_space_craft + k2.mass_space_craft
            * dt * 0.5;
        s3.time = sc.time + dt * 0.5;
        SpaceCraftState k3 = compute_derivatives(s3, t + dt * 0.5,
```

```
            active, fuel_rate);

        SpaceCraftState s4;
        s4.craft_pos = {sc.craft_pos.first + k3.craft_pos.first * dt,
                sc.craft_pos.second + k3.craft_pos.second * dt};
        s4.v_craft = {sc.v_craft.first + k3.v_craft.first * dt,
                sc.v_craft.second + k3.v_craft.second * dt};
        s4.mass_space_craft = sc.mass_space_craft + k3.mass_space_craft * dt;
        s4.time = sc.time + dt;
        SpaceCraftState k4 = compute_derivatives(s4, t + dt, active, fuel_rate);

        SpaceCraftState result;
        result.craft_pos = {sc.craft_pos.first + (k1.craft_pos.first + 2 * k2.craft_pos.first +
                2 * k3.craft_pos.first + k4.craft_pos.first) *
                dt / 6.0,
                sc.craft_pos.second + (k1.craft_pos.second + 2 * k2.craft_pos.second +
                2 * k3.craft_pos.second + k4.craft_pos.second) *
                dt / 6.0};
        result.v_craft = {sc.v_craft.first + (k1.v_craft.first + 2 * k2.v_craft.first +
                2 * k3.v_craft.first + k4.v_craft.first) *
                dt / 6.0,
                sc.v_craft.second + (k1.v_craft.second + 2 * k2.v_craft.second +
                2 * k3.v_craft.second + k4.v_craft.second) *
                dt / 6.0};
        result.mass_space_craft =
        sc.mass_space_craft + (k1.mass_space_craft + 2 * k2.mass_space_craft +
        2 * k3.mass_space_craft + k4.mass_space_craft) *
        dt / 6.0;
        result.time = sc.time + dt;

        return result;
}

SpaceCraftState SpaceCraftSolver::init_space_craft(double angle_deg, double mass) {
        double phi = angle_deg * M_PI / 180.0;
        State s0 = get_state_at_time(0.0);
        double vel = sqrt(Constants::G * Constants::M_PLANET / orbit_radius);

        return SpaceCraftState(
        {s0.planet_pos.first + orbit_radius * cos(phi),
                s0.planet_pos.second + orbit_radius * sin(phi)},
        {s0.v_planet.first - vel * sin(phi), s0.v_planet.second + vel * cos(phi)}, mass, 0.0);
}

double SpaceCraftSolver::try_flight(double angle, double mass, double total_time) {
        double fuel = mass - Constants::M0 - mass * Constants::PART_STRUCT_FRACTION;
```

```
        if (fuel <= 0.0) {
                return 1e20;
        }

        double fuel_rate = fuel / Constants::T_BURN;
        SpaceCraftState craft = init_space_craft(angle, mass);
        double t = 0.0, min_dist = 1e20;

        while (t < Constants::T_BURN) {
                State sys = get_state_at_time(t);
                double step = min(dt_burn, Constants::T_BURN - t);
                craft = runge_step(craft, t, step, true, fuel_rate);
                t += step;

                double d_planet = norm({craft.craft_pos.first - sys.planet_pos.first,
                        craft.craft_pos.second - sys.planet_pos.second}) -
                Constants::R_PLANET;
                if (d_planet <= 0.0) {
                        return 1e20;
                }
        }

        craft.mass_space_craft = Constants::M0;

        while (t < total_time) {
                State sys = get_state_at_time(t);
                double d_center = norm({craft.craft_pos.first - sys.sputnik_pos.first,
                        craft.craft_pos.second - sys.sputnik_pos.second});
                double d_surface = d_center - Constants::R_SPUTNIK;

                if (d_surface < min_dist) {
                        min_dist = d_surface;
                }
                if (d_surface <= 0.0) {
                        return -1.0;
                }

                double d_planet = norm({craft.craft_pos.first - sys.planet_pos.first,
                        craft.craft_pos.second - sys.planet_pos.second}) -
                Constants::R_PLANET;
                if (d_planet <= 0.0) {
                        return 1e20;
                }

                double step = min(dt_coast, total_time - t);
                craft = runge_step(craft, t, step, false, 0.0);
                t += step;
        }
        return min_dist;
}

void SpaceCraftSolver::find_best_fuel_angle_omp(double total_time) {
```

```
        cout << "parallel find fuel and angle" << endl;
        best_mass = 1e20;
        best_angle = 0.0;
        best_length = 1e20;

        int num_threads = omp_get_max_threads();

        vector<double> local_best_mass(num_threads, 1e20);
        vector<double> local_best_angle(num_threads, 0.0);
        vector<double> local_best_dist(num_threads, 1e20);
        vector<bool> local_found_collision(num_threads, false);

        #pragma omp parallel
        {
                int tid = omp_get_thread_num();
                bool found_collision = false;

                double mass_start = 39947.0;
                double mass_end = 49978.0;
                double mass_step = 0.01;
                int num_masses = static_cast<int>((mass_end - mass_start) / mass_step) + 1;

                #pragma omp for schedule(dynamic)
                for (int mass_idx = 0; mass_idx < num_masses; mass_idx++) {
                        if (found_collision) {
                                continue;
                        }

                        double mass = mass_start + mass_idx * mass_step;

                        double local_angle = 0.0;
                        double local_dist = 1e20;
                        bool collision_found = false;

                        for (double angle = 119.90; angle <= 119.99; angle += 0.01) {
                                double dist = try_flight(angle, mass, total_time);

                                if (dist <= 0.0) {
                                        local_dist = dist;
                                        local_angle = angle;
                                        collision_found = true;
                                        found_collision = true;
                                        break;
                                }

                                if (dist < local_dist) {
                                        local_dist = dist;
                                        local_angle = angle;
                                }
                        }

                        if (local_dist < local_best_dist[tid]) {
                                local_best_dist[tid] = local_dist;
                                local_best_angle[tid] = local_angle;
```

```
                                local_best_mass[tid] = mass;
                                local_found_collision[tid] = collision_found;
                        }

                        #pragma omp critical
                        {
                                cout << "mass " << mass << ", angle: " << local_angle << ", min_dist " << scientific
                                << local_dist << endl;
                        }
                }
        }

        for (int tid = 0; tid < num_threads; tid++) {
                if (local_best_dist[tid] < best_length) {
                        best_length = local_best_dist[tid];
                        best_angle = local_best_angle[tid];
                        best_mass = local_best_mass[tid];
                }
        }

        cout << "mass " << best_mass << ", angle " << best_angle << ", dist " << best_length << endl;
}

void SpaceCraftSolver::find_best_fuel_angle(double total_time) {
        best_mass = 1e20;
        best_angle = 0.0;
        best_length = 1e20;

        cout << "NOT parallel find fuel and angle" << endl;

        for (double mass = 39947.0; mass <= 39978.0; mass += 0.01) {
                double local_best_angle = 0.0;
                double local_best_dist = 1e20;

                for (double angle = 119.80; angle <= 119.00; angle += 0.01) {
                        double dist = try_flight(angle, mass, total_time);
                        if (dist <= 0.0) {
                                local_best_dist = dist;
                                local_best_angle = angle;
                                break;
                        }
                        if (dist < local_best_dist) {
                                local_best_dist = dist;
                                local_best_angle = angle;
                        }
                }

                if (local_best_dist < best_length) {
                        best_length = local_best_dist;
                        best_angle = local_best_angle;
                        best_mass = mass;
                }
```

```
                cout << "mass␣" << mass << ",␣angle␣" << local_best_angle << ",␣min_dist␣"
                << local_best_dist << endl;

                if (best_length <= 0.0) {
                        break;
                }
        }
}

void SpaceCraftSolver::run_computations(bool type_of_finding) {
        double total_time = planet_period + Constants::T_BURN;

        if (type_of_finding) {
                find_best_fuel_angle_omp(total_time);
                save_to_csv(total_time);
        } else {
                find_best_fuel_angle(total_time);
                save_to_csv(total_time);
        }
}

void SpaceCraftSolver::save_to_csv(double total_time) {
        ofstream file("../labs/lr1/trajectory_space_craft.csv");
        file << "time,x_spacecraft,y_spacecraft,x_sputnik,y_sputnik\n";

        double fuel = best_mass - Constants::M0 - best_mass * Constants::PART_STRUCT_FRACTION;

        cout << "fuel␣mass␣=␣" << fuel << ",␣best␣angle␣=␣" << best_angle
        << ",␣best␣total␣mass␣=␣" << best_mass << endl;
        if (fuel <= 0.0) {
                file.close();
                return;
        }

        double fuel_rate = fuel / Constants::T_BURN;
        SpaceCraftState craft = init_space_craft(best_angle, best_mass);
        double t = 0.0;
        bool collision_happened = false;

        State sys0 = get_state_at_time(0.0);
        file << t << "," << craft.craft_pos.first << "," << craft.craft_pos.second << ","
        << sys0.sputnik_pos.first << "," << sys0.sputnik_pos.second << "\n";

        while (t < Constants::T_BURN && !collision_happened) {
                double step = min(dt_burn, Constants::T_BURN - t);
                craft = runge_step(craft, t, step, true, fuel_rate);
                t += step;
                State sys = get_state_at_time(t);

                double dist_to_sputnik = norm({craft.craft_pos.first - sys.sputnik_pos.first,
                        craft.craft_pos.second - sys.sputnik_pos.second});

```

```
                if (dist_to_sputnik <= Constants::R_SPUTNIK) {
                        file << t << "," << craft.craft_pos.first << "," << craft.craft_pos.second << ","
                        << sys.sputnik_pos.first << "," << sys.sputnik_pos.second << "\n";
                        collision_happened = true;
                        break;
                }

                file << t << "," << craft.craft_pos.first << "," << craft.craft_pos.second << ","
                << sys.sputnik_pos.first << "," << sys.sputnik_pos.second << "\n";
        }

        if (!collision_happened) {
                craft.mass_space_craft = Constants::M0;

                while (t < total_time && !collision_happened) {
                        double step = min(dt_coast, total_time - t);
                        craft = runge_step(craft, t, step, false, 0.0);
                        t += step;
                        State sys = get_state_at_time(t);

                        double dist_to_sputnik = norm({craft.craft_pos.first - sys.sputnik_pos.first,
                                craft.craft_pos.second - sys.sputnik_pos.second});

                        if (dist_to_sputnik <= Constants::R_SPUTNIK) {
                                file << t << "," << craft.craft_pos.first << "," << craft.craft_pos.second << ","
                                << sys.sputnik_pos.first << "," << sys.sputnik_pos.second << "\n";
                                collision_happened = true;
                                break;
                        }

                        double dist_from_planet = norm({craft.craft_pos.first - sys.planet_pos.first,
                                craft.craft_pos.second - sys.planet_pos.second});
                        if (dist_from_planet > 10 * Constants::R_PLANET_SPUTNIK) {
                                break;
                        }

                        file << t << "," << craft.craft_pos.first << "," << craft.craft_pos.second << ","
                        << sys.sputnik_pos.first << "," << sys.sputnik_pos.second << "\n";
                }
        }

        file.close();
```

```
}
```

state.h

```
#pragma once

#include <iostream>
#include <utility>

using namespace std;

struct State {
        pair<double, double> sun_pos, planet_pos, sputnik_pos;
        pair<double, double> v_planet, v_sputnik;
        double time;

        State();
        State(pair<double, double>& sun_pos, pair<double, double>& planet_pos,
        pair<double, double>& sputnik_pos, pair<double, double>& v_planet,
        pair<double, double>& v_sputnik);
        State(const State& other);
};

struct SpaceCraftState {
        pair<double, double> craft_pos, v_craft;
        double mass_space_craft;
        double time;

        SpaceCraftState();
        SpaceCraftState(pair<double, double> pos, pair<double, double> vel, double mass, double t);
};
```

state.cpp

```
#include "state.h"

State::State() {
        sun_pos = {0.0, 0.0};
        planet_pos = {0.0, 0.0};
        sputnik_pos = {0.0, 0.0};
        time = 0;
}

State::State(pair<double, double>& sun_pos, pair<double, double>& planet_pos,
pair<double, double>& sputnik_pos, pair<double, double>& v_planet,
pair<double, double>& v_sputnik)
: sun_pos(sun_pos),
planet_pos(planet_pos),
sputnik_pos(sputnik_pos),
v_planet(v_planet),
v_sputnik(v_sputnik) {
        time = 0;
}

```

```
State::State(const State& other) {
        sun_pos = other.sun_pos;
        planet_pos = other.planet_pos;
        sputnik_pos = other.sputnik_pos;
        v_planet = other.v_planet;
        v_sputnik = other.v_sputnik;
        time = other.time;
}

SpaceCraftState::SpaceCraftState()
: craft_pos({0.0, 0.0}), v_craft({0.0, 0.0}), mass_space_craft(0.0),
    time(0.0) {}

SpaceCraftState::SpaceCraftState(pair<double, double> pos, pair<double,
    double> vel, double mass,
double t)
: craft_pos(pos), v_craft(vel), mass_space_craft(mass), time(t) {}
```

consts.cpp

```
#pragma once

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

namespace Constants {
        constexpr double M_SUN = 2e30;          // kg
        constexpr double M_PLANET = 1.9e27;     // kg
        constexpr double M_SPUTNIK = 4.8e22;    // kg

        constexpr double R_PLANET = 71500.0;          // km
        constexpr double R_SUN_PLANET = 780e6;        // km
        constexpr double R_SPUTNIK = 1561.0;          // km
        constexpr double R_PLANET_SPUTNIK = 671e3;    // km

        constexpr double V_PLANET = 13.0;    // km/s
        constexpr double V_SPUTNIK = 13.7;   // km/s

        constexpr double G = 6.67430e-11 / 1e9;

        constexpr double H = 2000.0;                      // km
        constexpr double T_BURN = 5100.0;                 // s
        constexpr double M0 = 40.0;                       // kg
        constexpr double V_EXP = 3.375;                   // km/s
        constexpr double PART_STRUCT_FRACTION = 0.001;    // mass fraction
}  // namespace Constants
```